

7/24/23, 2:49 PM

CodeProjectFileEditFindViewToolsEducationHelpRunProject index (stand)Debug

__init__.py

x

1 #print('The multiply package is now initialized!')

2 #import multiply.double, multiply.triple

3 __all__ = [

4 'double',

5 'triple',

6]

100% (6/6)

initializing.py

x

1 #import multiply

2 #print(multiply.double.double(3))

3 #print(multiply.triple.triple(3))

4

5 from multiply import *

6

7 print(dir())

8

100% (6/6)

https://docs.com/sandpan-dry/modules-and-packages/tree/code%2Finitializing.py

Code - Modules and Packages

Guide

Collapse

Package Initialization

Package Initialization

Previously, packages were described as a collection of modules. In reality, it is a little more complicated than that. Packages often include an initialization file entitled `__init__.py`. This file is run whenever a package is imported.

► **Did you notice?**

Package

</>

__init__.py

</>

Module

</>

Module

The **top** panel is the initialization file for the `multiply` package. Add the code below to `__init__.py`.

```
# top panel

print('The multiply package is now initialized!')
```

Next, in the **bottom** panel, import the `multiply` package. Do not add any other code. Now run your script.

```
# bottom panel

import multiply
```

TRY IT

The multiply package is now initialized!

Importing and Initializing

While the specific code that goes into a package's initialization file varies from case to case, initialization files can aid in the importing of packages. On the previous page, importing just the package and then calling the modules resulted in an error. That is because importing a module didn't do anything; there was no code in the initialization file.

Modify `__init__.py` so that importing the `multiply` module automatically imports the `double` and `triple` modules.

```
# top panel

import multiply.double, multiply.triple
```

In the **bottom** panel, import just the `multiply` module. Then call both the `double` and `triple` functions. Your script should work as expected now. This is because the initialization file for the package imported each module.

```
# bottom panel

import multiply

print(multiply.double.double(3))
print(multiply.triple.triple(3))
```

TRY IT

The multiply package is now initialized!
6
9

Another use case for the initialization file is when importing all of the contents of a package with the `*` wild card. First, let's start with the basic state of a Python script. Erase the code from the **top** panel.

```
# top panel
```

In the **bottom** panel, print the results of the `dir` function. This built-in function returns all available attributes and methods for all objects. Notice that even with no variable, function, or class declarations there are still attributes.

```
# bottom panel

print(dir())
```

TRY IT

['_annotations_', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__']

Now, import the entirety of the `multiply` package with `*`. Then print `dir` one more time. What do you think will happen?

```
# bottom panel

from multiply import *

print(dir())
```

TRY IT

['_annotations_', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__']

Nothing changed. The functions from the modules did not appear in the list. Unlike a module, the `*` operator does nothing when importing a package. This too can be changed in the initialization file.

In the **top** panel define the variable `__all__` to be a list of strings. Each element should be the name of a module in the `multiply` package.

```
# top panel

__all__ = [
    'double',
    'triple'
]
```

You do not need to make any changes to the code in the bottom panel. Running your script again should show you the `double` and `triple` functions.

TRY IT

['_annotations_', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'double', 'triple']

► **Using the `*` operator.**

Reading Question

Select all of the true statements for the `__init__.py` file. **Hint:** there is more than one correct answer.

☐ The `__init__.py` file is required when creating a package.

☐ The `__init__.py` file runs when the package is imported.

☐ The `__init__.py` file is optional but should be included when creating a package.

☐ The `__init__.py` file should perform initialization tasks like importing modules.

The correct answers are:

- The `__init__.py` file should perform initialization tasks like importing modules.
- The `__init__.py` file is optional but should be included when creating a package.
- The `__init__.py` file runs when the package is imported.

Packages do not require an initialization file, but this is not considered good form.